MQTT Tutorial between a local python script file and AWS IoT Core

This tutorial aims to demonstrate the fundamental mechanism of MQTT-based communication between an IoT device (in this case, a simulated temperature sensor) and the AWS IoT Core service.

Within AWS IoT Core, physical or virtual devices are represented as things. In the scope of this exercise, a thing named *mock-sensor-001* is created in the AWS Management Console. Upon successful registration of the thing, AWS IoT Core automatically generates a set of security credentials consisting of five files: a device certificate, a private key, a public key, and the Amazon Root CA certificates.

These credential files are then incorporated into a Python script that simulates sensor behavior by generating random temperature values at regular intervals. By configuring the script with the appropriate certificates and endpoint information, the script effectively acts as the registered thing which behaves as a legitimate IoT device or sensor.

A custom policy (*SensorPublishPolicy*) is subsequently created and attached to the thing's certificate. This policy explicitly authorizes the device to:

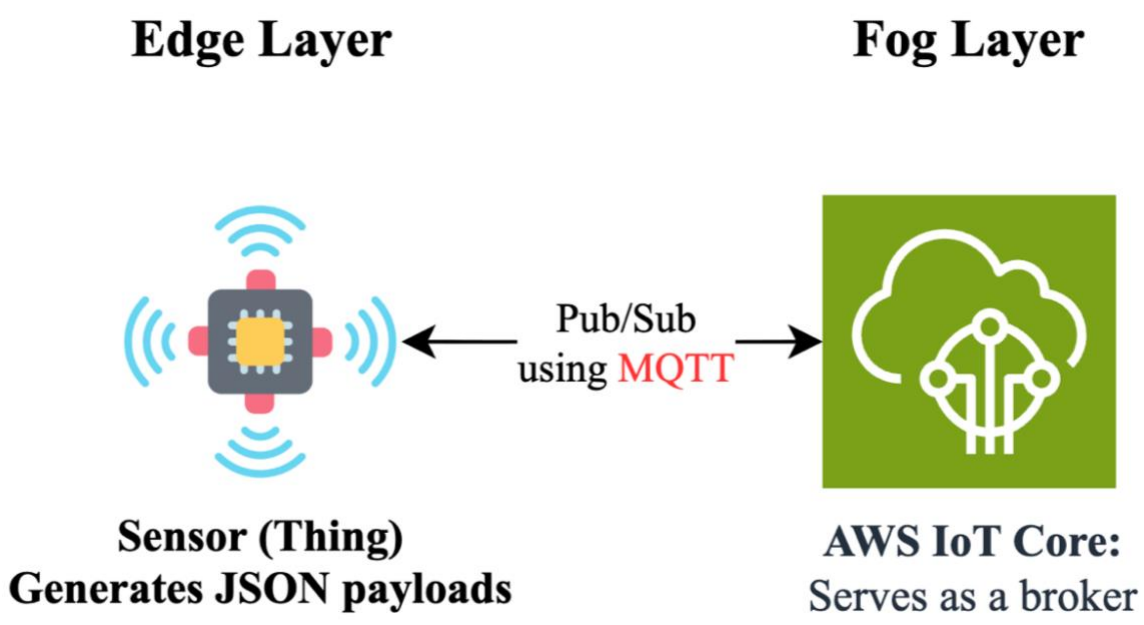
- connect to the AWS IoT Core MQTT broker using its own client ID,
- publish messages to a designated topic (sensors/mock-sensor-001/data).

Once the policy is correctly attached and the Python script is executed, the simulated sensor begins publishing temperature readings to AWS IoT Core via MQTT. These messages can be observed in real time by subscribing to the corresponding topic within the AWS IoT Core MQTT test client in the AWS Management Console.

This setup allows clear visualization and verification of bidirectional MQTT communication between an IoT device (the Python script) and the AWS IoT Core message broker.

The payloads transmitted by the device can subsequently be routed to other AWS services, such as Amazon DynamoDB for storage or Amazon S3 for long-term archiving, depending on the desired application architecture.

An architectural diagram is provided below to illustrate the overall flow and components involved in this tutorial.



Step 1 - Prepare Your Local Environment

1. Go to Visual Studio Code or other preferred IDEs
2. Create a project folder, called ‘fog-edge-tutorial’
3. Open a terminal inside Visual Studio Code.
4. Create and activate a virtual environment.

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

5. Install required Python packages

```
pip install awsiotsdk
pip install python-dotenv
pip install --upgrade pip
```

Step 2 - Create an IoT Thing in AWS IoT Core

1. Log into AWS Academy Learner Lab. Start your lab session. Open the AWS Management Console.
2. Go to AWS IoT Core → Manage → All devices → Things → Create things.
3. Choose Create single thing → Next.
4. Thing name: mock-sensor-001.
5. Leave other fields default → Next.
6. Device certificate: Choose Auto-generate a new certificate (recommended) → Next.
7. Policy: Skip and Create thing.

8. After creation, download all the certificate files and save them in a folder (e.g., fog-edge-tutorials/certs/) in VS Code. The total number of the certificate files should be 5.

Step 3 - Create an IoT Policy and attach to a thing.

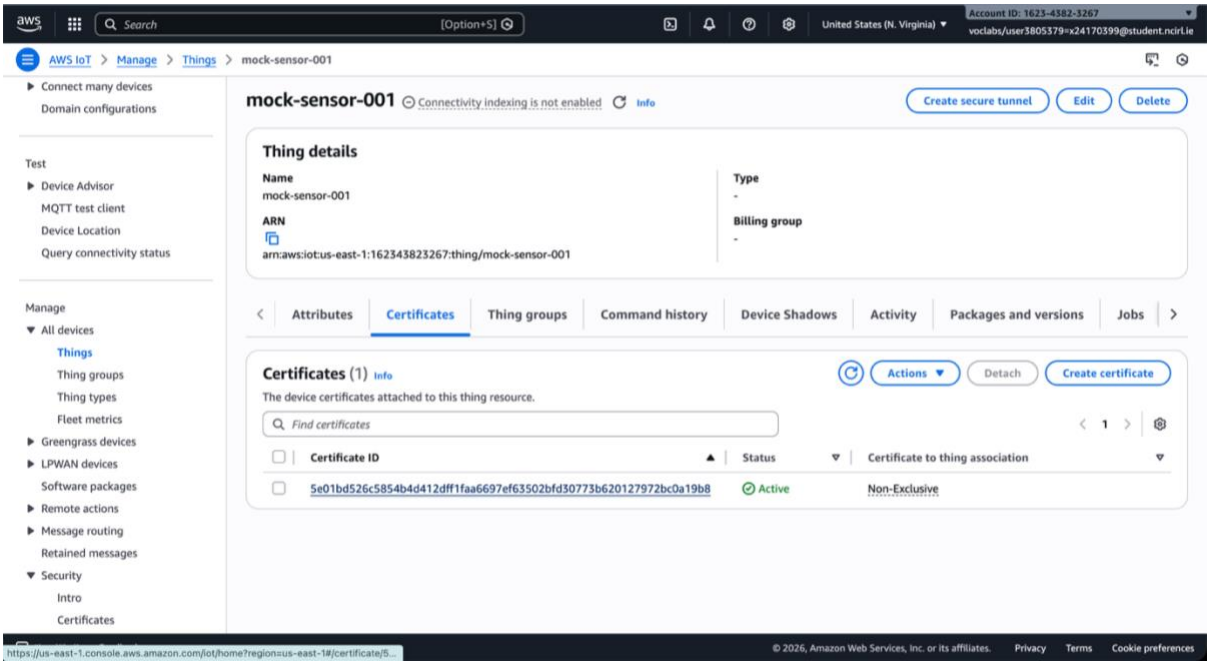
1. In the AWS Management Console, navigate to AWS IoT Core → Manage → Security → Policies.
2. Choose Create policy.
3. Policy name: SensorPublishPolicy.
4. In the policy editor, choose JSON view, then paste the following policy by replacing your thing's arn.

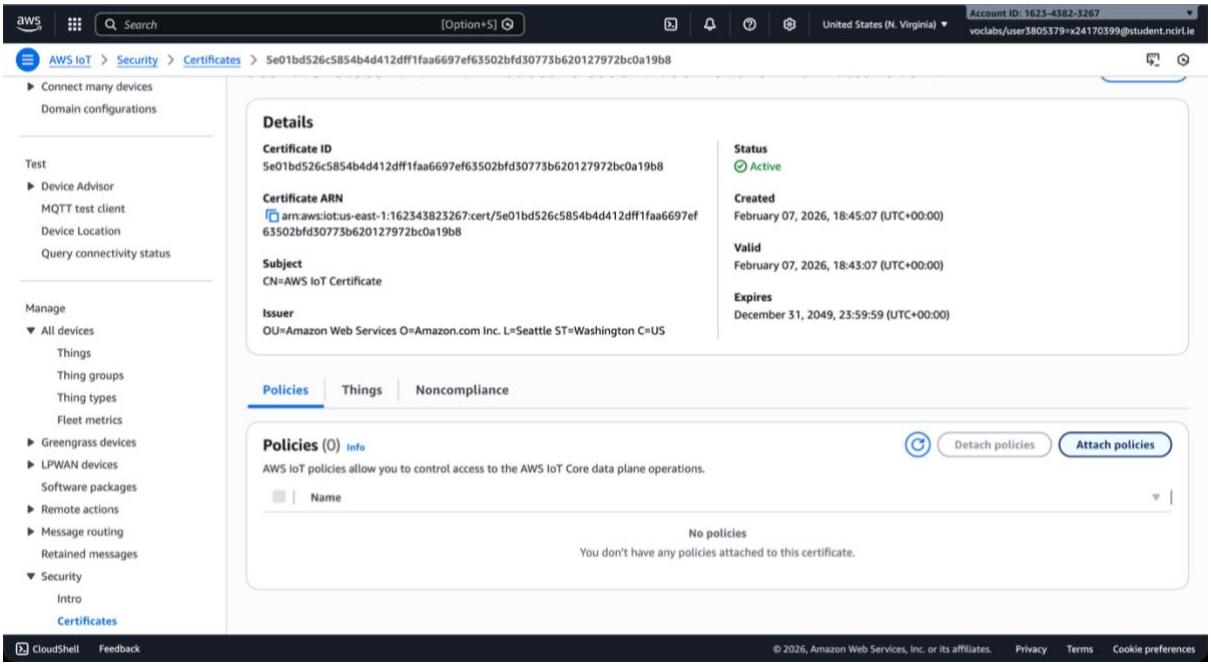


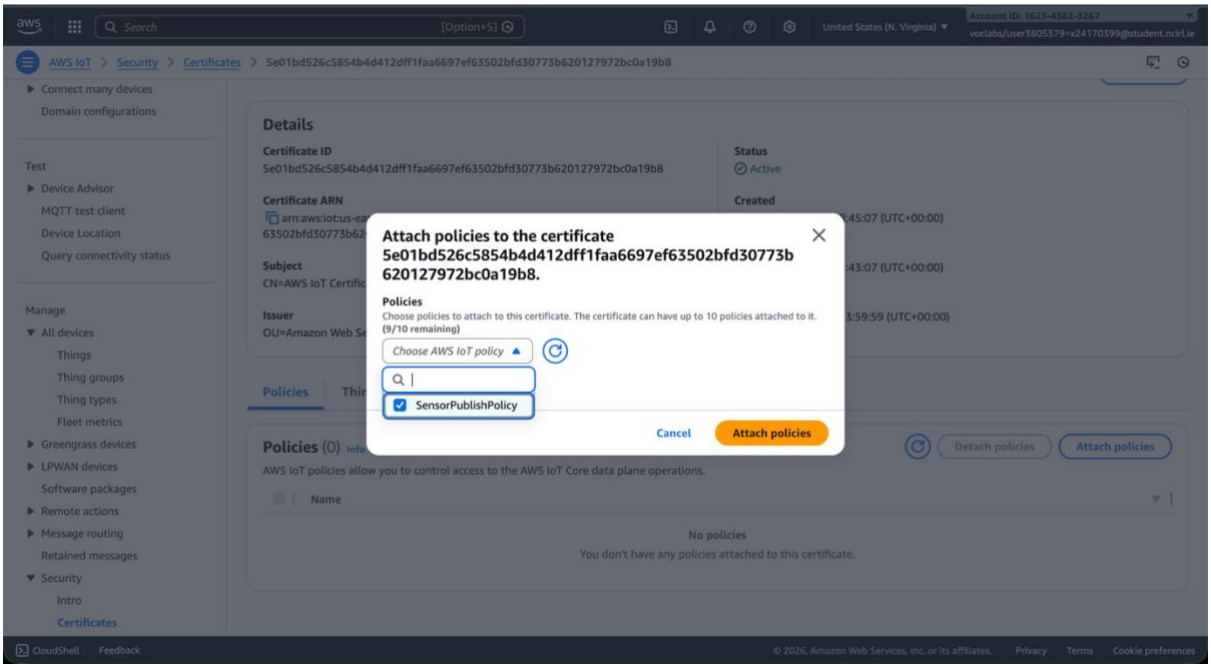
```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "iot:Connect",
            "Resource": "arn:aws:iot:us-east-1:162343823267:client/mock-sensor-001"
        },
        {
            "Effect": "Allow",
            "Action": "iot:Publish",
            "Resource": "arn:aws:iot:us-east-1:162343823267:topic/sensors/mock-sensor-001/data"
        },
        {
            "Effect": "Allow",
            "Action": "iot:Receive",
            "Resource": "arn:aws:iot:us-east-1:162343823267:topic/sensors/mock-sensor-001/data"
        },
        {
            "Effect": "Allow",
            "Action": "iot:Subscribe",
            "Resource": "arn:aws:iot:us-east-1:162343823267:topicfilter/sensors/mock-sensor-001/#"
        }
    ]
}
```

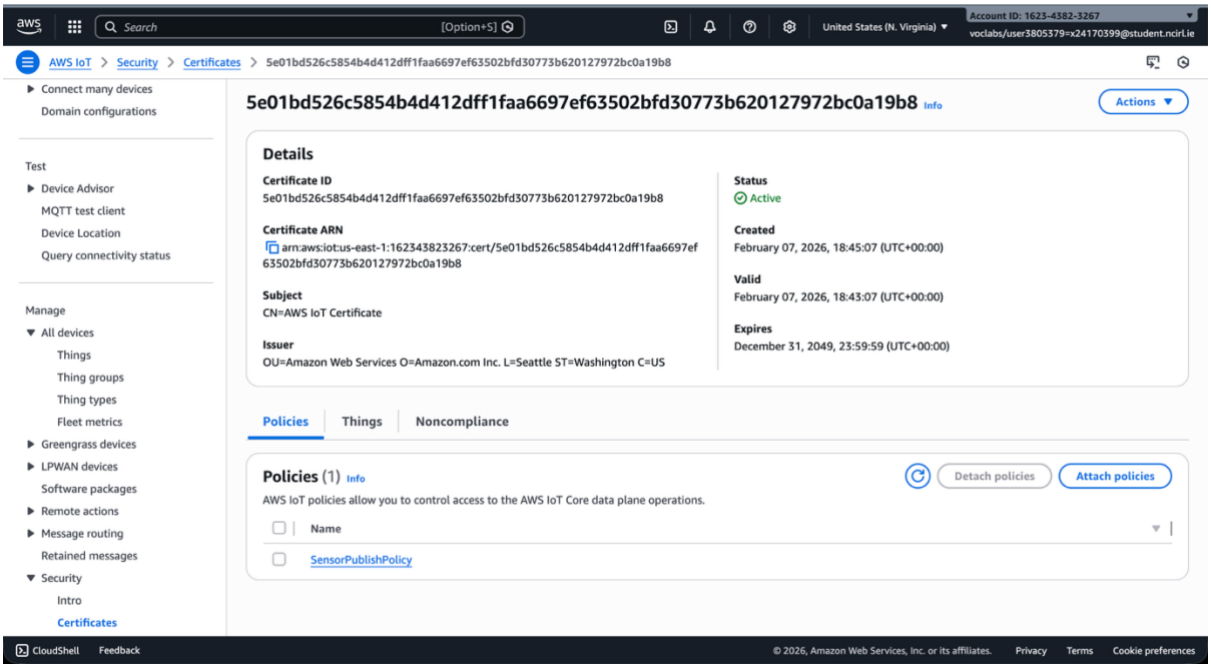
5. Press 'Create'.

6. Go back to Manage → All devices → Things → mock-sensor-001 → certificates → press the certificate → policies → attach policies.


Search
[Option+S]
United States (N. Virginia)
Account ID: 1623-4382-3267
voclabs/user3805379=x24170399@student.ncirl.ie
AWS IoT > Manage > Things > mock-sensor-001
Connect many devices
Domain configurations
Test
Device Advisor
MQTT test client
Device Location
Query connectivity status
Manage
All devices
Things
Thing groups
Thing types
Fleet metrics
Greengrass devices
LPWAN devices
Software packages
Remote actions
Message routing
Retained messages
Security
Intro
Certificates
mock-sensor-001
Connectivity indexing is not enabled
Info
Create secure tunnel
Edit
Delete
Thing details
Name
mock-sensor-001
Type
-
ARN
arn:aws:iot:us-east-1:162343823267:thing/mock-sensor-001
Billing group
-
Attributes
Certificates
Thing groups
Command history
Device Shadows
Activity
Packages and versions
Jobs
Certificates (1) Info
The device certificates attached to this thing resource.
Actions
Detach
Create certificate
Find certificates
Certificate ID
Status
Certificate to thing association
5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Active
Non-Exclusive
https://us-east-1.console.aws.amazon.com/iot/home?region=us-east-1#/certificate/5...
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Option+S]
United States (N. Virginia)
Account ID: 1623-4382-3267
voclabs/user3805379=x24170399@student.ncirl.ie
AWS IoT > Security > Certificates > 5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Connect many devices
Domain configurations
Test
Device Advisor
MQTT test client
Device Location
Query connectivity status
Manage
All devices
Things
Thing groups
Thing types
Fleet metrics
Greengrass devices
LPWAN devices
Software packages
Remote actions
Message routing
Retained messages
Security
Intro
Certificates
Details
Certificate ID
5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Status
Active
Certificate ARN
arn:aws:iot:us-east-1:162343823267:cert/5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Created
February 07, 2026, 18:45:07 (UTC+00:00)
Subject
CN=AWS IoT Certificate
Valid
February 07, 2026, 18:43:07 (UTC+00:00)
Issuer
OU=Amazon Web Services O=Amazon.com Inc. L=Seattle ST=Washington C=US
Expires
December 31, 2049, 23:59:59 (UTC+00:00)
Policies
Things
Noncompliance
Policies (0) Info
AWS IoT policies allow you to control access to the AWS IoT Core data plane operations.
Detach policies
Attach policies
Name
No policies
You don't have any policies attached to this certificate.
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Attach policies to the certificate 5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8.
Policies
Choose policies to attach to this certificate. The certificate can have up to 10 policies attached to it. (9/10 remaining)
Choose AWS IoT policy
SensorPublishPolicy
Cancel
Attach policies


5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Actions
Details
Certificate ID
5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Certificate ARN
arn:aws:iot:us-east-1:162343823267:cert/5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
Subject
CN=AWS IoT Certificate
Issuer
OU=Amazon Web Services O=Amazon.com Inc. L=Seattle ST=Washington C=US
Status
Active
Created
February 07, 2026, 18:45:07 (UTC+00:00)
Valid
February 07, 2026, 18:43:07 (UTC+00:00)
Expires
December 31, 2049, 23:59:59 (UTC+00:00)
Policies
Things
Noncompliance
Policies (1)
AWS IoT policies allow you to control access to the AWS IoT Core data plane operations.
Detach policies
Attach policies
Name
SensorPublishPolicy

Step 4 - Create the Python Script

1. Create file mqtt.py
2. Write the following code:

```
import time
import json
import random
from datetime import datetime
from awscrt import io, mqtt
from awsiot import mqtt_connection_builder

# ——————————————————————————————————
# AWS IoT Core Configuration - adjust to yours
# ——————————————————————————————————
ENDPOINT = "a15qobwkrxouvr-ats.iot.us-east-1.amazonaws.com"
CLIENT_ID = "mock-sensor-001"
TOPIC = f"sensors/{CLIENT_ID}/data"

# Certificate paths (relative to the script location)
CERT_PATH =
"certs/5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
-certificate.pem.crt"
KEY_PATH  =
"certs/5e01bd526c5854b4d412dff1faa6697ef63502bfd30773b620127972bc0a19b8
-private.pem.key"
CA_PATH   = "certs/AmazonRootCA1.pem"

# Publishing interval (seconds)
PUBLISH_INTERVAL = 10
# ——————————————————————————————————

# Minimal logging – use io.LogLevel.Error or .Debug only when
troubleshooting
io.init_logging(getattr(io.LogLevel, 'Fatal'), 'stderr')

# Build secure MQTT connection using mutual TLS authentication
print("Building MQTT connection...")
connection = mqtt_connection_builder.mtls_from_path(
    endpoint=ENDPOINT,
    cert_filepath=CERT_PATH,
    pri_key_filepath=KEY_PATH,
    ca_filepath=CA_PATH,
    client_id=CLIENT_ID,
    clean_session=True,
    keep_alive_secs=60,
    http_proxy_options=None
)

print("Connecting to AWS IoT Core...")
```

```
connect_future = connection.connect()
connect_future.result()           # Blocks until connected or raises
exception
print("Successfully connected!")

def publish_sensor_data():
    """Generate temperature reading and publish to AWS IoT."""
    payload = {
        "deviceId": CLIENT_ID,
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "temperature_C": round(random.uniform(18.0, 32.0), 1)
    }

    payload_json = json.dumps(payload)

    publish_future, packet_id = connection.publish(
        topic=TOPIC,
        payload=payload_json,
        qos=mqtt.QoS.AT_LEAST_ONCE
    )

    # Optional: wait for acknowledgment (useful for AT_LEAST_ONCE)
    publish_future.result()

    print(f"Published to '{TOPIC}': {payload}")


if __name__ == "__main__":
    print(f"Starting mock temperature sensor publisher (publishing
every {PUBLISH_INTERVAL}s)")
    print("Press Ctrl+C to stop\n")

    try:
        while True:
            publish_sensor_data()
            time.sleep(PUBLISH_INTERVAL)

    except KeyboardInterrupt:
        print("\nStopping...")

    finally:
        print("Disconnecting from AWS IoT Core...")
        try:
            if connection is not None:
                disconnect_future = connection.disconnect()
                disconnect_future.result()
                print("Disconnected successfully.")
        except Exception as e:
            print(f"Error during disconnect: {e}")
```

3. Make sure that you replace yours for ENDPOINT, CLIENT_ID, CERT_PATH, KEY_PATH and CA_PATH.

Step 5 - Test the Connection

1. Run the script in your VS Code terminal's virtual environment.

```
python mqtt.py
```

2. After running, the followings will be seen.



3. Open AWS IoT Core → left menu → Test → MQTT test client
4. Subscribe to: sensors/mock-sensor-001/data

